

Lab 0: Introduction to Network Protocols

Using Python Sockets

Part 0: Verify Python Is Installed

```
PS C:\Users\Brandon> py --version
Python 3.14.7
PS C:\Users\Brandon> py
Python 3.14.7 (tags/v3.14.7:823f032, Aug 5 2026, 10:51:32) [MSC v.1944 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>> print("Hello FSCT 8561")
Hello FSCT 8561
>>> exit9)
      File "<python-input-1>", line 1
        exit9)
              ^
SyntaxError: unmatched ')
>>> exit()
PS C:\Users\Brandon> |
```

Part 1: Your First Python Program

```
1 print("My first Python security program")
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\Brandon\Documents\School\Digital Forensics and Cybersecurity\FSCT 8561 - Security Applications\Labs\Lab 0> py .\lab0_test.py

My first Python security program

PS C:\Users\Brandon\Documents\School\Digital Forensics and Cybersecurity\FSCT 8561 - Security Applications\Labs\Lab 0>

Part 2 — Understanding Sockets

Reflection Questions

1. What is a Socket

A socket is a solution for an application to communicate over a network, typically to other devices.

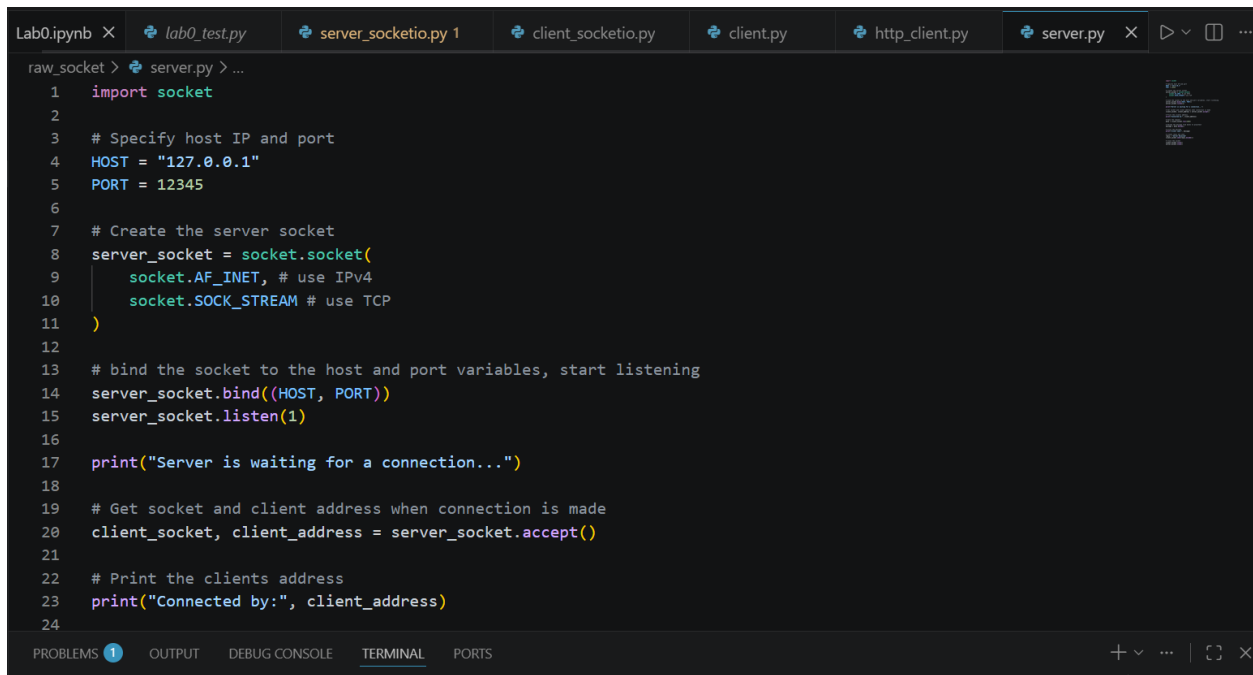
2. What does `AF\_INET` mean?

AF_INET specifies that the socket should use IPv4 when communicating over the network

3. What does `SOCK\_STREAM` mean?

SOCK_STREAM specifies that the socket should use a TCP connection when communicating over the network

Part 4 — Create a Basic TCP Server



The screenshot shows a JupyterLab environment with several open files: Lab0.ipynb, lab0_test.py, server_socketio.py 1, client_socketio.py, client.py, http_client.py, and server.py. The active file is server.py, which contains the following Python code:

```
raw_socket > server.py > ...
1  import socket
2
3  # Specify host IP and port
4  HOST = "127.0.0.1"
5  PORT = 12345
6
7  # Create the server socket
8  server_socket = socket.socket(
9      socket.AF_INET, # use IPv4
10     socket.SOCK_STREAM # use TCP
11 )
12
13 # bind the socket to the host and port variables, start listening
14 server_socket.bind((HOST, PORT))
15 server_socket.listen(1)
16
17 print("Server is waiting for a connection...")
18
19 # Get socket and client address when connection is made
20 client_socket, client_address = server_socket.accept()
21
22 # Print the clients address
23 print("Connected by:", client_address)
24
```

The interface also shows a bottom panel with tabs for PROBLEMS (1), OUTPUT, DEBUG CONSOLE, TERMINAL, and PORTS. The TERMINAL tab is currently selected.

```
Lab0.ipynb  lab0_test.py  server_socketio.py 1  client_socketio.py  client.py  http_client.py  server.py  X  ▾  [
raw_socket > server.py > ...
17  print('Server is waiting for a connection...')
18
19  # Get socket and client address when connection is made
20  client_socket, client_address = server_socket.accept()
21
22  # Print the clients address
23  print("Connected by:", client_address)
24
25  # Save the request
26  data = client_socket.recv(1024)
27
28  # Decode the message from bytes to plaintext
29  message = data.decode()
30
31  # print the message
32  print("Client says:", message)
33
34  # Create reply and send
35  reply = "Server Received"
36  client_socket.send(reply.encode())
37
38  # Close the socket
39  client_socket.close()
40  server_socket.close()
```

```
Windows PowerShell  X  Windows PowerShell  X  +  ▾  -  □  X
PS C:\Users\Brandon\Documents\School\Digital Forensics and Cyber security\FSC 8561 - Security Applications\Labs\Lab 0\raw_socket
> py server.py
Server is waiting for a connection...
|

PS C:\Users\Brandon\Documents\School\Digital Forensics and Cyber security\FSC 8561 - Security Applications\Labs\Lab 0\raw_socket
>
```

Part 5 — Create a Basic TCP Client

```
Lab0.ipynb  lab0_test.py  server_socketio.py 1  client_socketio.py  client.py  http_client.py  server.py

raw_socket > client.py > ...
1  import socket
2
3  # Create variables for IP and Port number
4  HOST = "127.0.0.1"
5  PORT = 12345
6
7  # create socket
8  client_socket = socket.socket(
9      socket.AF_INET, # Use IPv4
10     socket.SOCK_STREAM # Use TCP
11 )
12
13 # Connect to server using the host and the port
14 client_socket.connect((HOST, PORT))
15
16 message = "Hello Server"
17
18 # Send the message
19 client_socket.send(message.encode())
20
21
22 # save then print the response
23 response = client_socket.recv(1024)
24
```

```
Lab0.ipynb  lab0_test.py  server_socketio.py 1  client_socketio.py  client.py  http_client.py  server.py

raw_socket > client.py > ...
8  client_socket = socket.socket(
9      socket.AF_INET, # Use IPv4
10     socket.SOCK_STREAM # Use TCP
11 )
12
13 # Connect to server using the host and the port
14 client_socket.connect((HOST, PORT))
15
16 message = "Hello Server"
17
18 # Send the message
19 client_socket.send(message.encode())
20
21
22 # save then print the response
23 response = client_socket.recv(1024)
24
25 print("Server replied:", response.decode())
26
27 # close connection
28 client_socket.close()
```

```
Windows PowerShell
PS C:\Users\Brandon\Documents\School\Digital Forensics and Cyber security\FSC 8561 - Security Applications\Labs\Lab 0\raw_socket
> py server.py
Server is waiting for a connection...
Connected by: ('127.0.0.1', 54764)
Client says: Hello Server
PS C:\Users\Brandon\Documents\School\Digital Forensics and Cyber security\FSC 8561 - Security Applications\Labs\Lab 0\raw_socket
> |

Windows PowerShell
PS C:\Users\Brandon\Documents\School\Digital Forensics and Cyber security\FSC 8561 - Security Applications\Labs\Lab 0\raw_socket
> py .\client.py
Server replied: Message received
PS C:\Users\Brandon\Documents\School\Digital Forensics and Cyber security\FSC 8561 - Security Applications\Labs\Lab 0\raw_socket
> |
```

Part 6 — Modify the Client and Server

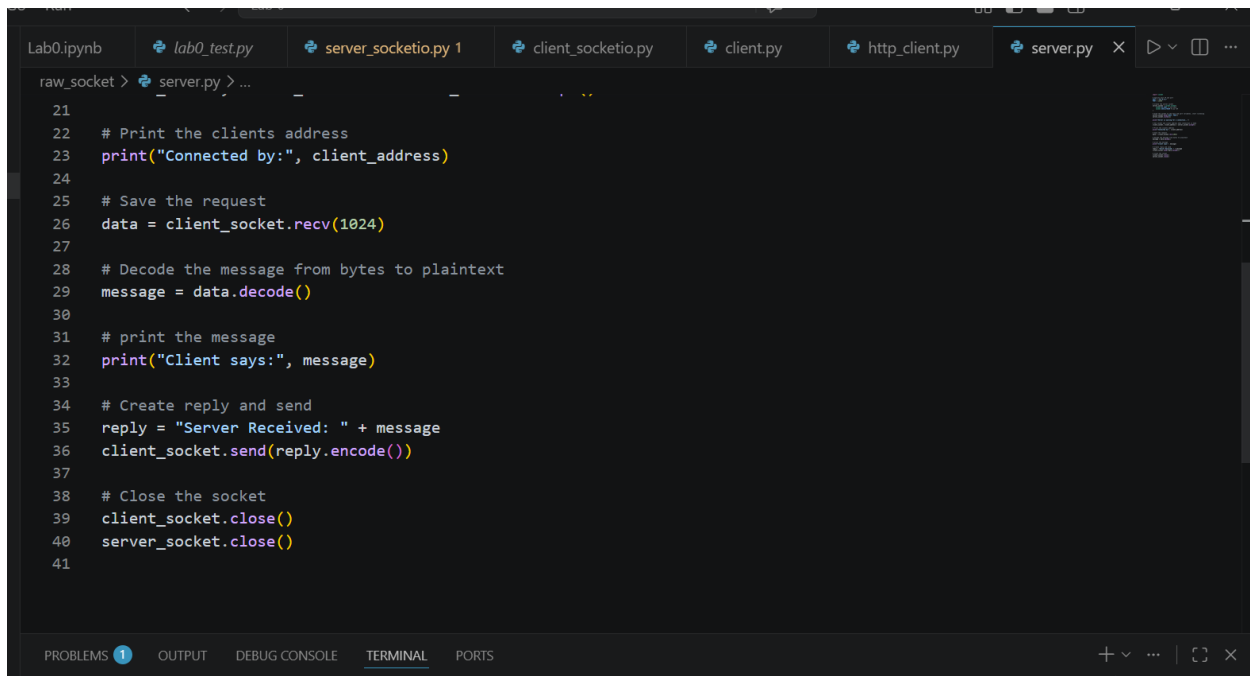
Client.py

```
Lab0.ipynb lab0_test.py server_socketio.py 1 client_socketio.py client.py x http_client.py server.py
raw_socket > client.py > ...
1 import socket
2
3 # Create variables for IP and Port number
4 HOST = "127.0.0.1"
5 PORT = 12345
6
7 # create socket
8 client_socket = socket.socket(
9     socket.AF_INET, # Use IPv4
10    socket.SOCK_STREAM # Use TCP
11 )
12
13 # Connect to server using the host and the port
14 client_socket.connect((HOST, PORT))
15
16 message = "Hello Server this is Brandon"
17
18 # Send the message
19 client_socket.send(message.encode())
20
21
22 # save then print the response
23 response = client_socket.recv(1024)
24
```

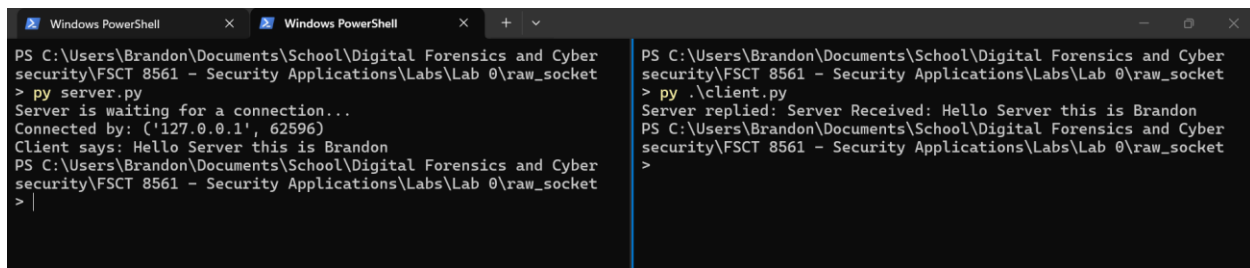
```
Run ... Lab 0
Lab0.ipynb lab0_test.py server_socketio.py 1 client_socketio.py client.py X http_client.py server.py
raw_socket > client.py > ...
6
7 # create socket
8 client_socket = socket.socket(
9     socket.AF_INET, # Use IPv4
10    socket.SOCK_STREAM # Use TCP
11 )
12
13 # Connet to server using the host and the port
14 client_socket.connect((HOST, PORT))
15
16 message = "Hello Server this is Brandon"
17
18 # Send the message
19 client_socket.send(message.encode())
20
21
22 # save then print the response
23 response = client_socket.recv(1024)
24
25 print("Server replied:", response.decode())
26
27 # close connection
28 client_socket.close()
```

Server.py

```
Lab0.ipynb lab0_test.py server_socketio.py 1 client_socketio.py client.py http_client.py server.py X
raw_socket > server.py > ...
1 import socket
2
3 # Specify host IP and port
4 HOST = "127.0.0.1"
5 PORT = 12345
6
7 # Create the server socket
8 server_socket = socket.socket(
9     socket.AF_INET, # use IPv4
10    socket.SOCK_STREAM # use TCP
11 )
12
13 # bind the socket to the host and port variables, start listening
14 server_socket.bind((HOST, PORT))
15 server_socket.listen(1)
16
17 print("Server is waiting for a connection...")
18
19 # Get socket and client address when connection is made
20 client_socket, client_address = server_socket.accept()
21
22 # Print the clients address
23 print("Connected by:", client_address)
24
```



```
raw_socket > server.py > ...
21
22 # Print the clients address
23 print("Connected by:", client_address)
24
25 # Save the request
26 data = client_socket.recv(1024)
27
28 # Decode the message from bytes to plaintext
29 message = data.decode()
30
31 # print the message
32 print("Client says:", message)
33
34 # Create reply and send
35 reply = "Server Received: " + message
36 client_socket.send(reply.encode())
37
38 # Close the socket
39 client_socket.close()
40 server_socket.close()
41
```



```
PS C:\Users\Brandon\Documents\School\Digital Forensics and Cyber security\FSC 8561 - Security Applications\Labs\Lab 0\raw_socket
> py server.py
Server is waiting for a connection...
Connected by: ('127.0.0.1', 62596)
Client says: Hello Server this is Brandon
PS C:\Users\Brandon\Documents\School\Digital Forensics and Cyber security\FSC 8561 - Security Applications\Labs\Lab 0\raw_socket
>

PS C:\Users\Brandon\Documents\School\Digital Forensics and Cyber security\FSC 8561 - Security Applications\Labs\Lab 0\raw_socket
> py .\client.py
Server replied: Server Received: Hello Server this is Brandon
PS C:\Users\Brandon\Documents\School\Digital Forensics and Cyber security\FSC 8561 - Security Applications\Labs\Lab 0\raw_socket
>
```

Part 7 — Connect to a Public Web Server

```
Lab0.ipynb  lab0_test.py  server_socketio.py 1  client_socketio.py  client.py  http_client.py X  server.py

raw_socket > http_client.py > ...
1  import socket
2
3  # Create socket and connect to example.com on port 80
4  client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
5  client.connect(("example.com", 80))
6
7  # Format the request (HTTP GET request)
8  request = (
9      "GET / HTTP/1.1\r\n"
10     "Host: example.com\r\n"
11     "Connection: close\r\n"
12     "\r\n"
13 )
14
15 # send message after it is encoded
16 client.send(request.encode())
17
18 # get the response and break after the server is finished
19 response = b""
20 while True:
21     data = client.recv(4096)
22     if not data:
23         break
24     response += data
25
26 # print the response
27 print(response.decode(errors="replace"))
28
29 # close the connection
30 client.close()
```

```
Windows PowerShell  X  Windows PowerShell  X  +  v  -  [  X
PS C:\Users\Brandon\Documents\School\Digital Forensics and Cybersecurity\FSC 8561 - Security Applications\Labs\Lab 0\socketio> py .\
http_client.py
HTTP/1.1 200 OK
Date: Thu, 10 Sep 2026 16:28:14 GMT
Content-Type: text/html
Transfer-Encoding: chunked
Connection: close
Server: cloudflare
Last-Modified: Wed, 02 Sep 2026 22:14:26 GMT
Allow: GET, HEAD
Accept-Ranges: bytes
Age: 11751
cf-cache-status: HIT
CF-RAY: a38fcdff0d1feef3-YVR

22f
<!doctype html><html lang="en"><head><title>Example Domain</title><link rel="icon" href="data:,"><meta name="viewport" content="width
=device-width, initial-scale=1"><style>body{background:#eee;width:60vw;margin:15vh auto;font-family:system-ui,sans-serif}h1{font-size
:1.5em}div{opacity:0.8}a:link,a:visited{color:#348}</style></head><body><div><h1>Example Domain</h1><p>This domain is for use in docu
mentation examples without needing permission. Avoid use in operations.</p><p><a href="https://iana.org/domains/example">Learn more</
a></p></div></body></html>

0

PS C:\Users\Brandon\Documents\School\Digital Forensics and Cybersecurity\FSC 8561 - Security Applications\Labs\Lab 0\socketio> |
```

Reflection Question 2

1. Which host did the client connect to?

The client connected to example.com

2. Which port was used?

Port 80

3. Why is port 80 relevant?

It is the designated port for HTTP traffic

4. What did the server send back?

It sent an HTML file for the website example.com

Part 8 - Challenge — Rebuild the Client-Server Application Using Socket.IO

Code from: <https://python-socketio.readthedocs.io/en/stable/intro.html#what-is-socket-io>

Client

```
Lab0.ipynb  lab0_test.py  server_socketio.py 1  client_socketio.py X  client.py  htt

socketio > client_socketio.py > ...
1  import socketio
2
3  # Create socket for client
4  sio = socketio.Client()
5
6  # Create connection event and send a message
7  @sio.event
8  def connect():
9      print('connection established')
10     sio.emit("my_message", {'hello': "HELLO SERVER ~ BRANDON WOO"})
11
12 # Create event for my_message, sends response to server
13 @sio.event
14 def my_message(data):
15     print("INFO - Incoming Message : ", data)
16     print("INFO - Sending Reply...")
17     sio.emit('my_response', {'response': 'I AM RESPONDING!! ~ BRANDON WOO'})
18
19 # Disconnect from server event
20 @sio.event
21 def disconnect():
22     print('disconnected from server')
23
24 # Begin connection to server on port 5000
25 sio.connect('http://localhost:5000')
26 sio.wait()
```

```
(venv) PS C:\Users\Brandon\Documents\School\Digital Forensics and Cybersecurity\FSCT 8561 - Security Applications\Labs\Lab 0\socketio> py .\client_socketio.py
connection established
INFO - Incoming Message :  {'my_response': 'HELLO CLIENT ~ BRANDON WOO'}
INFO - Sending Reply...
disconnected from server
```

Server:

```
socketio > server_socketio.py > ...
1  import eventlet
2  import socketio
3
4  # Create server socket
5  sio = socketio.Server()
6
7  # Create web server, not used in this program much, but was in the original code
8  app = socketio.WSGIApp(sio, static_files={
9      '/': {'content_type': 'text/html', 'filename': 'index.html'}
10 })
11
12 # Handle connection events
13 @sio.event
14 def connect(sid, environ):
15     print('connect ', sid)
16
17 # Handle my_message events, prints request and sends response
18 @sio.event
19 def my_message(sid, data):
20     print('message ', data)
21     print("INFO - Sending Reply...")
22
23     # Send Message to Client
24     sio.emit('my_message', {
25         'my_response': "HELLO CLIENT ~ BRANDON WOO"
26     }, to=sid)
27
28 # Handle "my_reponse" events
29 @sio.event
30 def my_response(sid, data):
31     print('Response from Client: ', data)
32
33 # Handle disconnection event
```

```
socketio > server_socketio.py > ...
8  app = socketio.WSGIApp(sio, static_files={
9      '/': {'content_type': 'text/html', 'filename': 'index.html'}
10 })
11
12 # Handle connection events
13 @sio.event
14 def connect(sid, environ):
15     print('connect ', sid)
16
17 # Handle my_message events, prints request and sends response
18 @sio.event
19 def my_message(sid, data):
20     print('message ', data)
21     print("INFO - Sending Reply...")
22
23     # Send Message to Client
24     sio.emit('my_message', {
25         'my_response': "HELLO CLIENT ~ BRANDON WOO"
26     }, to=sid)
27
28 # Handle "my_reponse" events
29 @sio.event
30 def my_response(sid, data):
31     print('Response from Client: ', data)
32
33 # Handle disconnection event
34 @sio.event
35 def disconnect(sid):
36     print('disconnect ', sid)
37
38 # Start program and listen on port 5000
39 if __name__ == '__main__':
40     eventlet.wsgi.server(eventlet.listen(('', 5000)), app)
```

```
(venv) PS C:\Users\Brandon\Documents\School\Digital Forensics and Cybersecurity\FSCT 8561 - Security Applications\Labs\Lab 0\socketio> py .\server_socketio.py
C:\Users\Brandon\Documents\School\Digital Forensics and Cybersecurity\FSCT 8561 - Security Applications\Labs\Lab 0\socketio\server_socketio.py:1: EventletDeprecationWarning:
Eventlet is deprecated. It is currently being maintained in bugfix mode, and
we strongly recommend against using it for new projects.
```

If you are already using Eventlet, we recommend migrating to a different framework. For more detail see <https://eventlet.readthedocs.io/en/latest/asyncio/migration.html>

```
import eventlet
(27008) wsgi starting up on http://0.0.0.0:5000
(27008) accepted ('127.0.0.1', 54207)
127.0.0.1 -- [10/Sep/2026 10:34:38] "GET /socket.io/?transport=polling&EI0=4&t=1789061676.2189975 HTTP/1.1" 200 300 0.000844
(27008) accepted ('127.0.0.1', 54209)
connect HCS2oVIimKQUU760AAAB
message {'hello': 'HELLO SERVER ~ BRANDON WOO'}
INFO - Sending Reply...
Response from Client: {'response': 'I AM RESPONDING!! ~ BRANDON WOO'}
disconnect HCS2oVIimKQUU760AAAB
127.0.0.1 -- [10/Sep/2026 10:55:22] "GET /socket.io/?transport=websocket&EI0=4&sid=Setl5p69_rW3Qw9xAAAA&t=1789061678.2690215 HTTP/1.1" 200 0 1242.337029
```

Security Reflection

1. What could happen if a server accepts connections from anyone?

It could accept connections from malicious actors that intend to do harm to the server or other people connecting to it

2. Is the Lab 0 message encrypted?

No, because the message was sent over HTTP which is not encrypted. To encrypt, it should be sent over HTTPS

3. Could network traffic be observed by another system?

Yes, if there was a man in the middle attack, an attacker could read out plaintext messages since they were not encrypted.

4. Why should servers validate input?

So that the server can determine if input is malicious. For example, an attacker could perform a SQL injection if data was not validated when it is received

5. Why might authentication be useful?

You can keep a list of authenticated clients who are allowed to send messages to the server. If a client fails authentication, the server should deny connections to ensure security.

6. Suggest two ways to make this application more secure.

Implement HTTPS for encryption of data as well as authentication to prevent unauthorized clients from connecting to the server.

Github Repo: <https://github.com/bwoo14/FSCT-8561-Labs.git>